Table 1: Build of AWFY at `-O3`, median of five rounds in seconds (range of the total).

Configuration	Total	Analysis	Compile	Memory, GiB
`graal`	24.1 (23.1–24.8)	4.4	7.4	5.3
`fields`	25.4 (24.8–25.6)	6.3	7.7	5.4
added	+1.3 (+5 %)	+1.9 (+43 %)	+0.3 (+4 %)	+0.1
`graal` on allocation sites	30.4 (30.3–32.6)	11.8	7.4	5.5
`objects` on allocation sites	33.3 (32.6–34.1)	13.7	7.8	5.5
added	+2.9 (+10 %)	+1.9 (+16 %)	+0.4 (+5 %)	0.0

Table 2: Build of the larger programs, median of three builds in seconds (range of the totals), by the build of `graal`.

Program	`graal`	`fields`	Added	Analysis	Loads
Spring PetClinic	128.0 (127.0–130.0)	172.0 (170.0–173.0)	+44.0 (+34 %)	+42.6	−0.9 %
PetClinic	116.0 (112.0–119.0)	146.0 (146.0–151.0)	+30.0 (+26 %)	+33.8	−0.9 %
play-scala	67.0 (67.0–68.0)	82.0 (79.0–84.0)	+15.0 (+22 %)	+14.9	−0.8 %
dropwizard	63.0 (62.0–63.0)	73.0 (72.0–80.0)	+10.0 (+16 %)	+12.2	−0.9 %
javac	62.0 (61.0–64.0)	76.0 (69.0–78.0)	+14.0 (+23 %)	+7.9	−3.1 %
spring-hello	62.0 (60.0–64.0)	73.0 (71.0–74.0)	+11.0 (+18 %)	+11.4	−0.8 %
micronaut-shopcart	61.0 (60.0–62.0)	72.0 (71.0–75.0)	+11.0 (+18 %)	+10.4	−0.6 %
micronaut-similarity	56.5 (56.2–57.3)	66.0 (64.0–67.0)	+9.5 (+17 %)	+10.0	−0.7 %
ktor	55.8 (55.2–57.6)	65.0 (64.0–67.0)	+9.2 (+16 %)	+10.1	−0.4 %
micronaut-hello	55.5 (55.5–56.3)	66.0 (65.0–67.0)	+10.5 (+19 %)	+9.6	−0.7 %
helidon	46.0 (44.9–46.5)	52.8 (51.0–52.8)	+6.8 (+15 %)	+5.9	−0.8 %
vertx	37.3 (36.0–37.9)	41.1 (40.6–43.5)	+3.8 (+10 %)	+5.1	−0.8 %
plain hello world	29.9 (29.1–30.2)	32.1 (31.7–32.5)	+2.2 (+7 %)	+2.6	–

Table 3: Build time of Native Image on the default analysis and with allocation sites, in seconds, and the ratio.

Program (builds)	Default	Allocation sites	Times
AWFY (five rounds)	24.1	30.4	1.3
javac (three rounds)	67	288	4.3
PetClinic (one build)	120	1,830	15
Spring PetClinic (one build)	138	out of memory	–
play-scala	72	771	10.7
dropwizard	69	537	7.8
micronaut-shopcart	66	647	9.8
spring-hello	65	567	8.7
ktor	63	364	5.8
micronaut-hello	62	550	8.9
micronaut-similarity	59	460	7.8
helidon	50	377	7.5
vertx	42	265	6.3
plain hello world	33	99	3.0

Table 4: Our steps in the `fields` builds, and in the `objects` build of AWFY, in seconds (median).

	Steps					
Program	Facts	Write	Tool	Sum	Stage added	Apply
Spring PetClinic	12.3	17.6	12.7	42.5	+42.6	2.3
PetClinic	9.8	14.9	8.9	33.6	+33.8	1.7
play-scala	6.0	5.1	3.4	14.5	+14.9	1.1
dropwizard	6.0	3.2	2.3	11.5	+12.2	0.6
javac	4.8	1.4	1.2	7.4	+7.9	0.5
spring-hello	5.6	3.4	2.5	11.5	+11.4	0.7
micronaut-shopcart	4.7	3.7	2.6	11.0	+10.4	0.9
micronaut-similarity	4.2	2.9	2.1	9.3	+10.0	0.7
ktor	5.0	2.5	1.9	9.4	+10.1	0.6
micronaut-hello	4.2	2.9	2.1	9.2	+9.6	0.7
helidon	3.1	1.8	1.2	6.0	+5.9	0.3
vertx	3.3	1.0	0.9	5.2	+5.1	0.3
plain hello world	1.9	0.5	0.5	2.9	+2.6	0.1
AWFY	1.5	0.2	0.3	1.9	+1.9	0.1
AWFY on allocation sites, `objects`	1.5	0.2	0.3	2.0	+1.9	0.3

Table 5: Run time of AWFY at `-O3`, default analysis, in percent against the first `graal` build (negative is faster).

Benchmark	`graal`, ms	A/A	other `graal` builds	`fields`, four builds
Queens	23.3	+0.4	−1.8 to −1.5	**−27.8 to −27.5**
Sieve	84.9	−0.8	−5.3 to −4.3	−11.3 to −8.1
Storage	42.3	−0.4	−0.2 to +0.3	−4.8 to −2.7
Permute	27.1	+1.1	−1.1 to +0.4	+7.2 to +8.5
List	39.5	0.0	+1.3 to +2.3	+1.5 to +4.1
Richards	97.6	+0.6	+3.3 to +9.9	+1.7 to +11.5
DeltaBlue	14.4	−0.3	+0.2 to +0.9	−0.1 to +1.7
Havlak	65.7	+0.6	+1.5 to +2.5	−2.1 to +2.4
Other six	27–83	−0.1 to +0.3	−0.4 to +1.3	−0.5 to +1.2
Total	699.6	+0.1	0.0 to +1.1	−1.4 to −0.1

Table 6: Loads left in the program's methods; the percentage is the change against `graal` on the same analysis.

	Default analysis		Allocation sites		
Program	`graal`	`fields`	`graal`	`fields`	`objects`
`LoadStoreElimination`	40	27 (−32.5 %)	43	30 (−30.2 %)	30 (−30.2 %)
`Signatures`	16	12 (−25.0 %)	16	12 (−25.0 %)	12 (−25.0 %)
`Aliasing`	16	14 (−12.5 %)	18	16 (−11.1 %)	13 (−27.8 %)
`Allocation`	9	7 (−22.2 %)	9	7 (−22.2 %)	7 (−22.2 %)
Four small programs	81	60 (−25.9 %)	86	65 (−24.4 %)	62 (−27.9 %)
AWFY, `-O2`	943	903 (−4.2 %)	954	914 (−4.2 %)	914 (−4.2 %)
AWFY, `-O3`	2,565	2,511 (−2.1 %)	–	–	–

Table 7: The default-analysis loads of the small programs, `graal` to `fields`, by kind of method. *Controls* are the methods named `...Control`: a call in them may write the field, or a reference may alias another, so they must keep their loads. *Unchanged* are the loads of the other methods that do not change (helpers, `main`, inlined monitor code, and in `Signatures` the three methods that cross a monitor, a native call, and a `volatile` access). *Reduced* are the loads of the methods that lose loads. The three add up to the totals of the default analysis in the table above.

Program	Controls	Unchanged	Reduced
`LoadStoreElimination`	16 → 14	10	14 → 3
`Signatures`	–	12	4 → 0
`Aliasing`	5 → 4	9	2 → 1
`Allocation`	–	0	9 → 7
Four small programs	21 → 18	31	29 → 11 (−62.1 %)

Table 8: The ceiling on AWFY at `-O3`, default analysis: the loads left and the run time in percent against `graal`, one build of each; the loads of the four builds timed together differ from these by at most one.

	Loads	In loops	Total	Queens	Permute	Richards
`graal`	2,565	937	–	–	–	–
`fields`	2,511 (−2.1 %)	904	+0.4	−27.9	+6.5	+13.7
`ceiling`	2,393 (−6.7 %)	859	−1.5	−26.7	−7.6	−0.7

Table 9: Control flow of AWFY at `-O3`, default analysis, summed over the 185 methods the images compile.

	`graal`	`fields`
Blocks	21,197	21,201
Control-flow edges	28,997	29,003
Ifs	8,515	8,514
Loops	602	599
Methods with other control flow	–	9

Table 10: The methods whose control flow changes most, `graal` to `fields`. The other five of the nine change by at most 12 edges each.

Method	Blocks	Edges	Ifs	Loops
`Queens.placeQueen`	85 → 207	119 → 299	39 → 97	1 → 0
`Permute.benchmark`	106 → 15	152 → 19	49 → 5	1 → 0
`Permute.permute`	31 → 16	41 → 19	13 → 6	–
`Queens.queens`	120 → 126	169 → 176	–	2 → 1

Table 11: The loads that `fields` removes from AWFY at `-O3`, by where they were. A load that left a loop and appears outside it was hoisted, peeled, or unrolled: the counts cannot tell which. The numbers are changes in the count of loads, `graal` to `fields`.

Loads	Change
in loops	−33
of these, left the loop and appear outside it	5
of these, gone (forwarded, merged, or folded)	28
entered a loop	0
outside loops (the 5 included)	−21
all loads	−54

Table 12: Methods of AWFY (of 459) freed from killing everything when the kill of the named effects is left out of the kill sets (export of 22 October).

Left out	Methods freed
identity hash codes, synchronization, unresolved calls, or other effects, each alone	0
class initialization alone	12
identity hash codes, class initialization, and synchronization together	12
… and unresolved calls	12
… and every other effect	199

Table 13: Evidence that the images are correct.

Check	Result
Output	every image of this chapter prints exactly what the JVM prints
Controls that must fail	a method claimed to write nothing gives 171 instead of 177 in `LoadStoreElimination` at `-O3`, so a false claim does not pass unseen
Native Image's own tests	with the feature and `-J-ea -J-esa`, its 23 JUnit images build and all 289 tests pass; the assertions found three defects that the examples did not: disabled flows asked for their objects, calls killed the initialization state, and a JSON escape failed on half a surrogate pair